Informe de práctica de la semana 8

Integración del sistema de reservas de hotel

UNAP · Ingeniería de Sistemas · SIS206 POO II · 2026 II
S8-001 · 19 de octubre de 2026 · Informe revisado

1 Descripción del sistema

Hotel Puno es un sistema académico de reservas para recepción y administración. Permite registrar datos de prueba, cotizar habitaciones, confirmar reservas, consultar disponibilidad y anular solicitudes. Integra los conceptos de la Unidad I en Python y C++17.

La integración aprobó 13 pruebas Python y 12 C++17 y produjo demostraciones equivalentes. Se verificaron reservas válidas, fechas inválidas, solapamientos, intervalos adyacentes, búsquedas sin resultado y cancelaciones. Las notificaciones se simulan en memoria; los datos se conservan durante la ejecución.

Componente	Evidencia incluida
Repositorio	Estructura y comandos reproducibles; historial local de preparación.
Modelo UML	Clases completas, dos secuencias y un diagrama de estados.
Demostración	Reserva válida, conflicto, búsqueda fallida, error, orden y cancelación.
Informe	Tres páginas con las cinco secciones exactas de EVI1.
Defensa	Guion de diez minutos y ficha de revisión para completar.

Caso de uso demostrado

La reserva R1 asigna la habitación doble 102 al huésped H1 del 1 al 3 de noviembre de 2026. Dos noches a S/ 150 producen un total de S/ 300. Un segundo intervalo superpuesto se rechaza mediante Result. Al anular R1 vuelven a quedar disponibles las tres habitaciones del conjunto de prueba.

Ejecución reproducible desde EVI1: python main.py y python verificar.py. Las fuentes se organizan en python, cpp/include, cpp/src y tests. El informe está en docs/informe_evi1.pdf; los resultados y comandos se conservan en evidencias.

2 Decisiones de diseño

Herencia y polimorfismo

HabitacionSimple, HabitacionDoble y Suite son habitaciones y mantienen el contrato de precio positivo. La demostración utiliza una colección heterogénea, con precios de 100, 150 y 200. La reserva usa un huésped y una habitación; no hereda de ellos.

Interfaces y composición

ServicioReservas recibe CalculadoraTarifa e INotificadorHotel. Se puede sustituir la cotización o el notificador sin cambiar la entidad. Las notificaciones se simulan en memoria. La máquina de estados pertenece a la reserva y valida los cambios antes de registrarlos.

Errores y contratos

ErrorHotel agrupa errores de datos, estados y disponibilidad. Las búsquedas y conflictos esperados retornan Result. Se rechazan fechas inválidas y se mantiene intacta la colección si una creación falla.

Colecciones y disponibilidad

Los repositorios usan dict o unordered_map para búsquedas por ID; las listas o vectores ordenados son vistas de consulta. Los intervalos de alojamiento son semiabiertos: una reserva que sale el día en que otra entra no se solapa.

3 Modelo UML

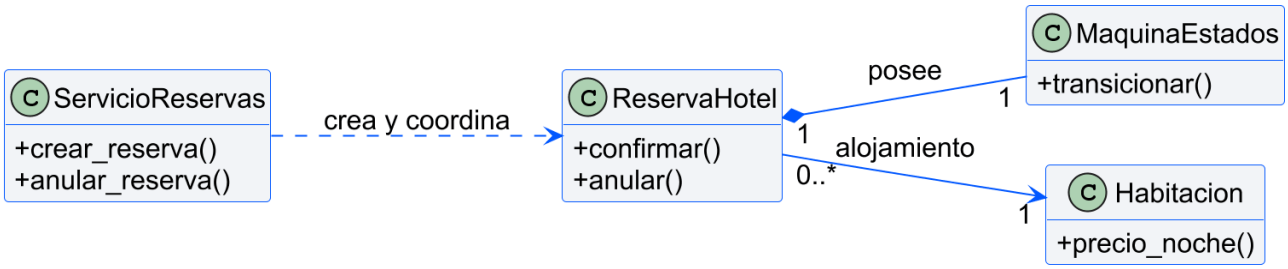


Figura 1 Detalle del modelo. ServicioReservas crea y coordina la reserva; ReservaHotel posee su máquina de estados y referencia una habitación. El rombo lleno indica composición. Una habitación puede participar en cero o más reservas a lo largo del tiempo; cada reserva tiene una habitación.

El modelo completo añade huésped, subtipos, interfaces y repositorios. Se acompaña de dos secuencias y un diagrama de estados en docs/uml. La tabla de trazabilidad relaciona sus mensajes con los métodos del código.

4 Reflexión y aprendizajes

Coordinación de disponibilidad y estados

La decisión más delicada fue derivar la disponibilidad desde intervalos y reservas activas. Una cancelación libera el intervalo sin modificar una segunda bandera. Las fechas usan intervalos semiabiertos: una salida el día 3 permite otra entrada el día 3. Las pruebas comprueban tanto el solapamiento como esa adyacencia.

Mejoras previstas para otra iteración

El siguiente paso sería incorporar persistencia transaccional y definir una política de recuperación cuando falle una notificación. Para aplicar SOLID en la Unidad II, las estrategias de tarifa pueden crecer mediante nuevas implementaciones de CalculadoraTarifa; una fachada podría coordinar los casos de uso sin trasladar reglas a la interfaz de usuario.

Aprendizaje que debe demostrarse

La defensa debe explicar por qué los tipos de habitación conservan el contrato de la base, cómo se sustituye una tarifa inyectada y por qué una transición inválida no modifica el historial. El guion y la ficha de revisión permiten preparar esa explicación con los archivos reales de la entrega.

Codex asistió en el desarrollo, las pruebas y la documentación. Esta reflexión describe decisiones de la implementación y no una experiencia personal atribuida al estudiante. Los commits locales corresponden a la preparación actual; faltan los datos del estudiante, GitHub, revisión de pares y sustentación.

5 Referencias

Fowler, M. (2018). *Refactoring Improving the Design of Existing Code* (2.ª ed.). Addison Wesley. https://martinfowler.com/books/refactoring.html

Object Management Group. (2017). *OMG Unified Modeling Language Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1/

Python Software Foundation. (s. f.). *abc — Abstract Base Classes*. Python 3.12 documentation. https://docs.python.org/3.12/library/abc.html

Zanabria Gálvez, A. H. (2026). *Guía de práctica de laboratorio de la semana 8*. SIS206 POO II, Universidad Nacional del Altiplano. Material del curso.